What the frozen forests split on

A reading of the two random forests frozen on 12 October 2026. The trees were opened and counted. They were not retrained.

Stage 2 asks whether memory utilization or power is moving. If it is, the window is called training. Fallback asks whether memory jumped in the first 30 samples of the run. If it did, the window is called training. Two of fallback's later questions point the other way: a GPU that is nearly full, and a memory bus that is busy while the processors are not. The tables list each question, where it sits in the tree, and which side is called training.

What we did

The detector has a network rule and two forests on the same 174 GPU features. The rule fires when a node's mean send+receive rate is at least 1 MiB/s and the two directions are balanced (the smaller is at least 0.8 of the larger). Stage 2 was fit only on windows where that rule fires: 3,019 training windows and 939 others, out of 3,958. Fallback was fit on every honest window: 10,059 windows, 244 jobs, 42 families. A window is called training when the forest that is allowed to speak has probability at least 0.5. Adversarial jobs were not in the fit.

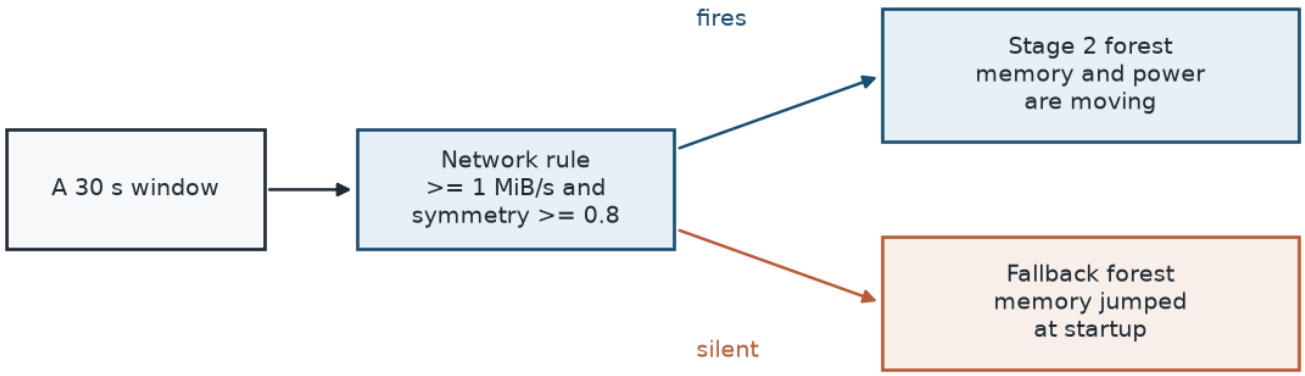


Figure 1. Which forest is asked. Once the rule fires, fallback is not consulted.

Each forest is 200 trees, minimum 2 samples in a leaf, 13 features drawn at each split, class weights balanced, seed 0. The stage 2 and fallback sections list the questions those trees ask. A cut quoted there is the typical threshold, the median of many split points weighted by how much mix each split removed. "Where the impurity goes" says how the bar chart counts those questions.

The two forests are not the same size

Stage 2	Fallback
Mean depth 4.8, deepest 9. About 10 leaves per tree. 162 of 174 features are split on at least once.	Mean depth 13.1, deepest 18. About 78 leaves per tree. 164 of 174 features are split on at least once.

Stage 2 is a short tree. Fallback is a real one. They also cut on different facts, which is the rest of this note.

Where the impurity goes

Skip ahead to Stage 2 if you want the questions first. This section is only how the bar lengths were counted.

Each tree is a stack of yes/no questions. A question looks at one column ("is the spread of memory utilization above 1.2?") and sends the window left or right.

Before the question, training and not-training are mixed together. Gini measures that mix. With two classes it is 2p(1-p), where p is the share of the weight that is training. It equals 0.5 when the two classes are even, and 0 when every window on that side is the same class. A useful question is one that makes the mix drop. The drop at a node is

```
N * Gini - N_left * Gini_left - N_right * Gini_right
```

N is not a count of windows. The forest was fit with balanced class weights, so the scarcer class pulls harder than its share of the rows, and N is that weight, already stored on the node. We add the drop over every question in all 200 trees and give the sum to the column the question used. Divide by the total drop. That percentage is the bar. It is the same number sklearn calls feature importance. Recomputing it from the nodes matches sklearn to 0.0002.

The bars are not 174 separate columns. The spread, the range, and the interquartile range of memory utilization are one measurement saved three times. Drawn separately, one fact would look like three. Columns that come from the same place are added:

Bar	Which columns go into it
Memory	The name contains "mem": utilization, megabytes in use, and the jump in the first 30 samples.
Power and temperature	The name contains "power" or "temperature".
PCIe	The name starts with "pcie".
Compute	Whatever is left: GPU utilization, SM clock, idle fraction, tensor activity.
Video	Encoder and decoder. Those columns do not vary here, so the share is zero.

A column scores only at questions where the tree actually picked it. Each question is shown 13 of the 174 columns, so a twin that was not shown gets nothing for that question. And stage 2's memory bar of 61% does not mean memory caused the call. The startup jump and the utilization wobble are both inside that bar. Figure 2 is the total. The two tables below split that bar into the actual questions.

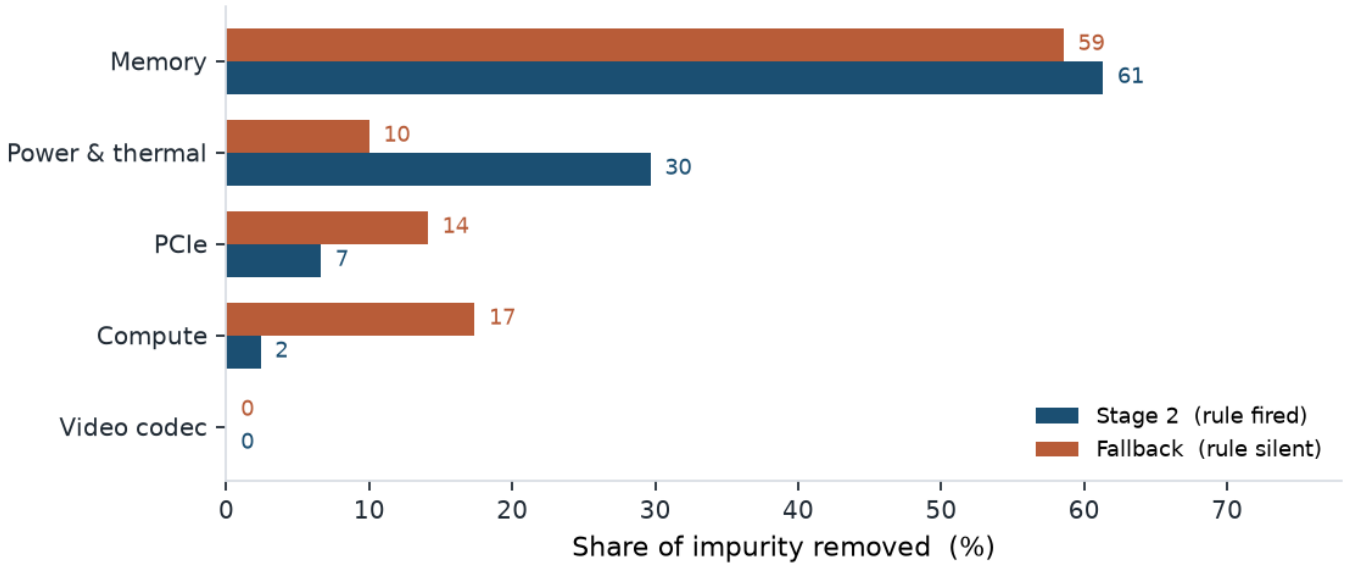


Figure 2. Of all the mix a forest removed, the share removed by questions about each source. Stage 2's memory bar is 61%: that share of the mix it removed came from a split on a column whose name contains "mem".

Stage 2 puts 61% on memory and 30% on power and temperature. PCIe is 7%. Compute, which is where GPU utilization and SM clock live, is 2%, and none of that is at the root. GPU-utilization mean is rank 105 and removes essentially no impurity.

Fallback is still 59% memory, but the memory it uses is the allocation at the start of the run. Compute rises to 17%, PCIe to 14%, and power falls to 10%.

Stage 2: is memory or power moving?

Stage 2 trained only on windows where the network rule had already fired: 3,019 training windows and 939 others. The forest is short, average depth 4.8, about 10 leaves per tree. The not-training windows in that fit are HPC jobs with balanced traffic and a flat GPU trace. Inference was not in the fit. A serving window whose memory utilization moves lands on the training side of every question below, which is why stage 2 never grew a leaf that recognizes inference.

The question	Where in the tree	Above the cut
Is the spread of memory utilization above 1.2 percentage points?	First question in 17 of 200 trees. 88% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.95 higher than below it, so the two sides are nearly pure.
Is the range of that same utilization above 4.2 percentage points? Same measurement, stored again.	First question in 17 of 200 trees. 92% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.95 higher than below it, so the two sides are nearly pure.
Is the interquartile range of that same utilization above 1.3 percentage points? Same measurement, a third copy.	First question in 10 of 200 trees. 94% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.90 higher than below it, so the two sides are nearly pure.
Is the spread of power above 9 W?	First question in 13 of 200 trees. 92% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.86 higher than below it, so the two sides are nearly pure.
Is GPU memory in use above 10 GB?	First question in 11 of 200 trees. 74% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.88 higher than below it, so the two sides are nearly pure.
Does memory take longer than 16 samples to reach its plateau? A slow climb is above this cut.	First question in 12 of 200 trees. 81% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.90 higher than below it, so the two sides are nearly pure.

Table 1. Stage 2's questions. "Above the cut" is the side where the measured value is higher. The training-share difference is the tree's own class mix on the high side minus the mix on the low side, weighted by how much mix the split removed. A difference of 1 would mean one side is all training and the other side is all not-training. Depth 0 is the first question in a tree.

The startup jump that dominates fallback ranks between 32 and 64 here, so stage 2 barely asks it. GPU-utilization mean ranks 105. Spread, range, and interquartile range together are the first question in 44 of the 200 trees. Each split is shown only 13 of the 174 columns, so a measurement that is the first question about as often as it is offered is one the tree actually prefers.

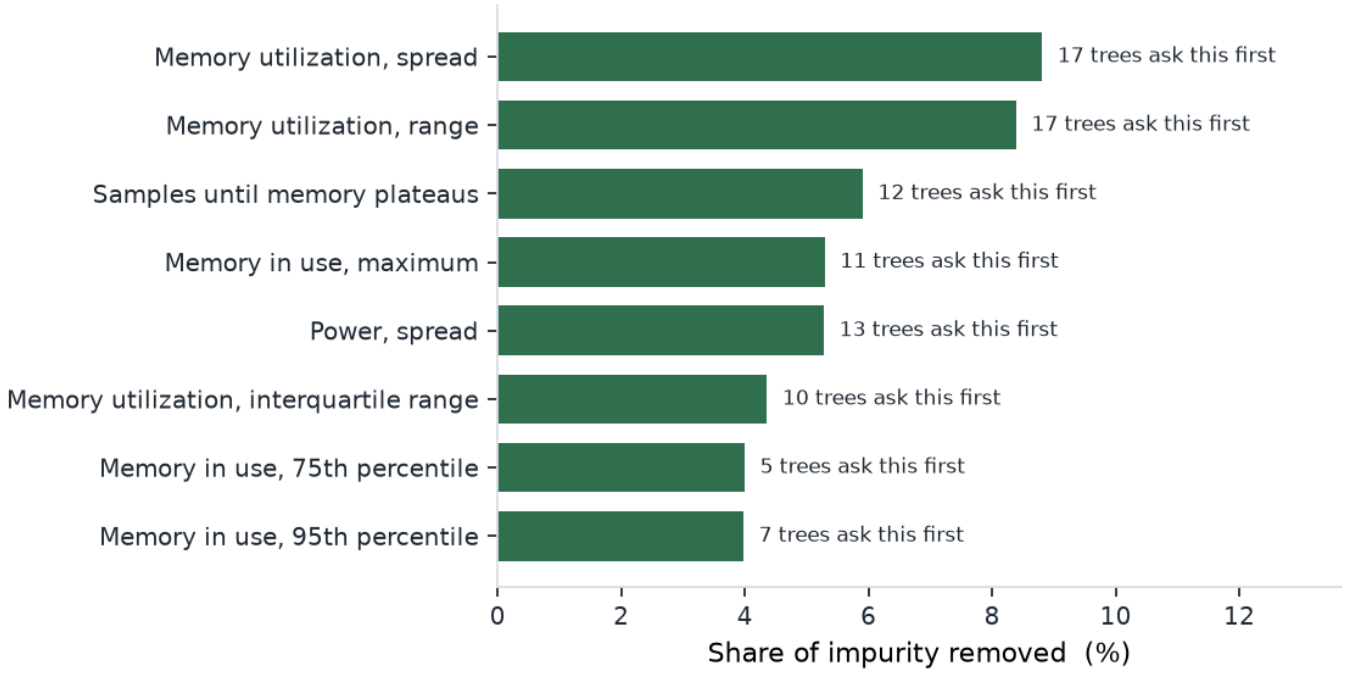


Figure 3. The same stage 2 columns, sized by how much mix they remove. Green means above the cut is training. Several bars are one measurement: spread, range, and the interquartile range of memory utilization.

Fallback: did memory jump at startup?

Fallback trained on every honest window. It is a deeper forest, average depth 13.1, about 78 leaves per tree. Its first questions read the first 30 samples of the run, and only as far as the current window is allowed to see. Once those 30 samples have passed, later windows of the same job all see the same jump. The three startup rows below are that one jump written three ways. Together they are 20% of the mix fallback removes, and 37% of the weight of first questions. The code comment says the feature was written to catch optimizer state allocated at startup, on the order of 6 GB.

The question	Where in the tree	Above the cut
Did memory rise by more than 4.9 GB in those first 30 samples?	First question in 11 of 200 trees. Most of the mix it removes comes from one or two questions later.	Training. Above the cut, the training share is 0.70 higher than below it.
Is the peak in those samples more than 11 times the starting value? Same jump, as a ratio.	First question in 15 of 200 trees. 45% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.68 higher than below it.
Is that rise more than 4% of the GPU's memory? Same jump, as a fraction of capacity.	First question in 13 of 200 trees. 36% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.63 higher than below it.
Is memory utilization minus GPU utilization above -23 points? Above this, the memory bus is busy relative to the processors.	First question in 12 of 200 trees. 36% of the mix this column removes comes from that first question.	Not training. Above the cut, the training share is 0.62 lower than below it.
Is GPU memory in use above 68 GB? Stage 2's cut on this same column is 10 GB, and there the high side is training.	First question in 8 of 200 trees. Most of the mix it removes comes from one or two questions later.	Not training. Above the cut, the training share is 0.36 lower than below it.
Is the range of PCIe transmit above 859 MB/s?	First question in 13 of 200 trees. 48% of the mix this column removes comes from that first question.	Training. Above the cut, the training share is 0.57 higher than below it.

Table 2. Fallback's questions, read the same way as Table 1. The fourth and fifth rows point the other way: above the cut is not training.

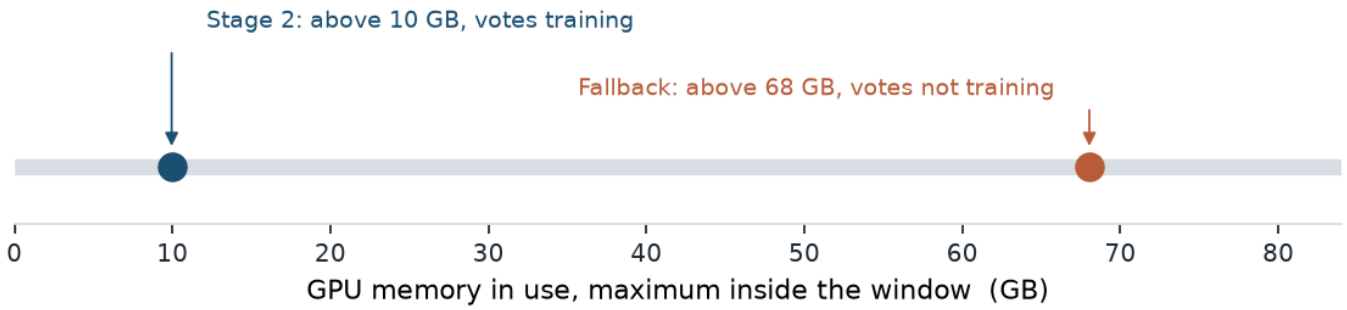


Figure 5. One column, two forests. Stage 2, which only saw balanced-traffic windows, calls training above about 10 GB. Fallback, which also saw large serving jobs, calls not-training above about 68 GB. Both cuts are weighted medians of split points.

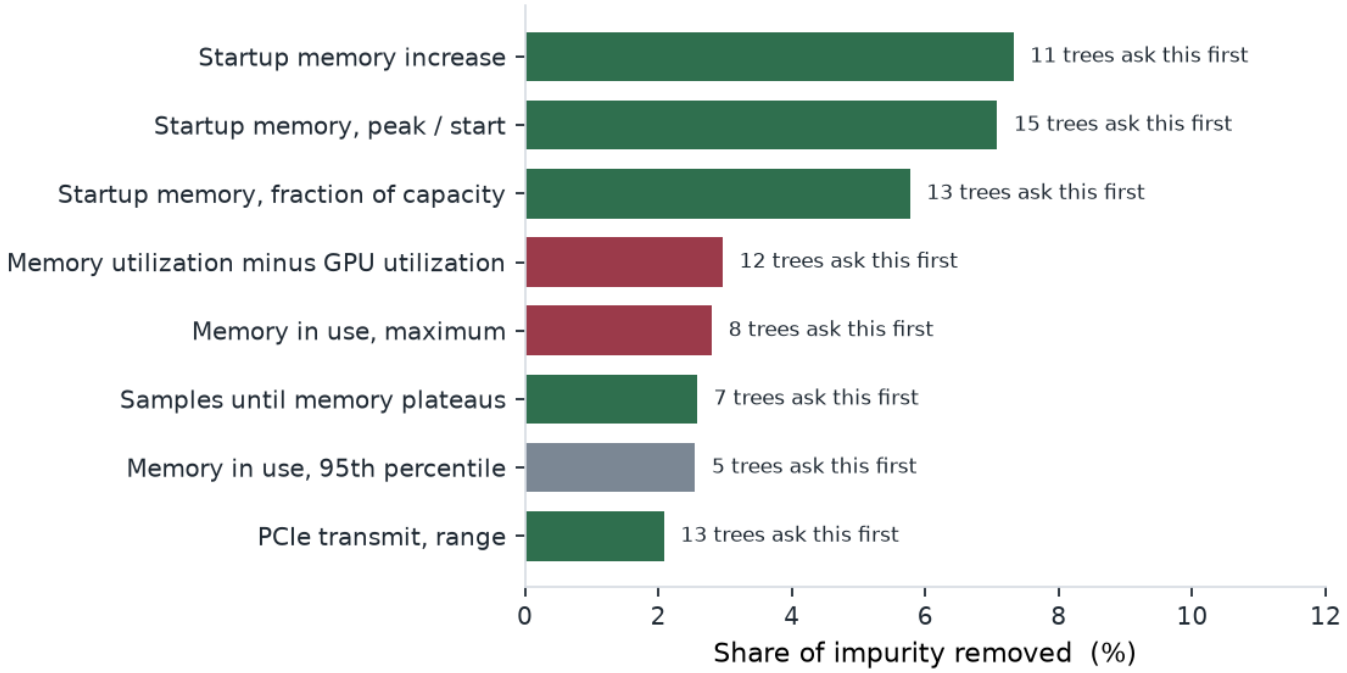


Figure 4. The same fallback columns, sized by how much mix they remove. Green: above the cut is training. Red: above the cut is not training. Gray: the two sides barely differ. The first three bars are one jump.

GPU utilization is not one of these questions. In fallback its mean ranks 19, and the typical cut is 99.7%, so only a pinned GPU registers. The training share above that cut is 0.52 higher than below it, a weak split. In stage 2 this column ranks 105.

What that does to the gate

On the 93 Nebius jobs already scored with this frozen detector, the pattern matches the cuts. Those jobs were not used to choose features or to move the 0.5 cutoff. Probabilities below are the mean over the job's windows.

Workload	Rule	Stage 2	Fallback	Call
Training, IB off (iboff_flat)	silent	not asked	0.65	training
Training, no gradient collective (nosync_flat)	silent	not asked	0.62 to 0.66	training
LoRA, one node, IB off (lora_1n8_iboff)	silent	not asked	0.53 to 0.54	training on 85% of windows
Qwen serving plus a fake all-reduce (qwen_fake_ar)	fires	0.90 to 0.93	0.30 to 0.37	training

Workload	Rule	Stage 2	Fallback	Call
OLMoE across nodes (olmoe_ep_mixed)	fires	0.89 to 0.90	0.11 to 0.13	training
Split prefill/decode plus a return message (disagg_echo)	fires on most windows	0.91 to 0.95	0.35 to 0.47	training

Table 3. Orange rows are decided by fallback, because the rule is silent. Blue rows are decided by stage 2, because the rule fired. Fallback's probability on the blue rows is below 0.5 and is ignored.

The orange rows are training processes whose cross-node all-reduce is missing. Fallback sees the startup allocation and calls them training. That is the feature doing what it was built for. The blue rows are inference with balanced traffic on the wire. Stage 2 sees memory moving, the way it saw training, and calls them training. On every one of the 1,032 windows in this score where the rule fired, stage 2's probability was at least 0.62. It never vetoed.

Encoder utilization, decoder utilization, and the variation of the memory clock are never split on in either forest. Those columns are constant in the training corpus.

What this note is not

A question on memory spread means the saved trees separate the two classes there. It does not mean that changing the spread would turn a window into training. We did not run that experiment.

Spread, range, and the interquartile range are one measurement stored three times. Figure 2 adds them, with the startup jump, into the memory bar. Tables 1 and 2 list the questions one by one. That is where stage 2's memory questions turn out to be about utilization moving, and fallback's turn out to be about the jump at startup.

Nothing here trains a new forest, adds the October campaigns, or moves the 0.5 cutoff. Shuffling a column on held-out jobs was not done either; that would need the training table. The questions and the probabilities are the freeze from 12 October 2026. The script is classifier_analysis/frozen_interp.py. Run it with scikit-learn 1.8 so the saved trees load unchanged.